# Smart Contract
# Security Audit Report

# Table Of Contents

**1 Executive Summary**

**2 Audit Methodology**

**3 Project Overview**

3.1 Project Introduction

3.2 Vulnerability Information

**4 Code Overview**

4.1 Contracts Description

4.2 Visibility Description

4.3 Vulnerability Summary

**5 Audit Result**

**6 Statement**

# 1 Executive Summary

On 2026.03.30, the SlowMist security team received the edgeX team's security audit application for EdgeDistributor, developed the audit plan according to the agreement of both parties and the characteristics of the project, and finally issued the security audit report.

The SlowMist security team adopts the strategy of "white box lead, black, grey box assists" to conduct a complete security test on the project in the way closest to the real attack.

The test method information:

| Test method | Description |
| --- | --- |
| Black box testing | Conduct security tests from an attacker's perspective externally. |
| Grey box testing | Conduct security testing on code modules through the scripting tool, observing the internal running status, mining weaknesses. |
| White box testing | Based on the open source code, non-open source code, to detect whether there are vulnerabilities in programs such as nodes, SDK, etc. |

The vulnerability severity level information:

| Level | Description |
| --- | --- |
| Critical | Critical severity vulnerabilities will have a significant impact on the security of the DeFi project, and it is strongly recommended to fix the critical vulnerabilities. |
| High | High severity vulnerabilities will affect the normal operation of the DeFi project. It is strongly recommended to fix high-risk vulnerabilities. |
| Medium | Medium severity vulnerability will affect the operation of the DeFi project. It is recommended to fix medium-risk vulnerabilities. |
| Low | Low severity vulnerabilities may affect the operation of the DeFi project in certain scenarios. It is suggested that the project team should evaluate and consider whether these vulnerabilities need to be fixed. |
| Weakness | There are safety risks theoretically, but it is extremely difficult to reproduce in engineering. |

| Level | Description |
| --- | --- |
| Suggestion | There are better practices for coding or architecture. |

# 2 Audit Methodology

The security audit process of SlowMist security team for smart contract includes two steps:

- Smart contract codes are scanned/tested for commonly known and more specific vulnerabilities using automated analysis tools.
- Manual audit of the codes for security issues. The contracts are manually analyzed to look for any potential problems.

Following is the list of commonly known vulnerabilities that was considered during the audit of the smart contract:

| Serial Number | Audit Class | Audit Subclass |
| --- | --- | --- |
| 1 | Overflow Audit | - |
| 2 | Reentrancy Attack Audit | - |
| 3 | Replay Attack Audit | - |
| 4 | Flashloan Attack Audit | - |
| 5 | Race Conditions Audit | Reordering Attack Audit |
| 6 | Permission Vulnerability Audit | Access Control Audit |
| | | Excessive Authority Audit |
| 7 | Security Design Audit | External Module Safe Use Audit |
| | | Compiler Version Security Audit |
| | | Hard-coded Address Security Audit |

| Serial Number | Audit Class | Audit Subclass |
|---|---|---|
| | | |
| | | Fallback Function Safe Use Audit |
| | | Show Coding Security Audit |
| | | Function Return Value Security Audit |
| | | External Call Function Security Audit |

| Serial Number | Audit Class | Audit Subclass |
|---|---|---|
| 7 | Security Design Audit | Block data Dependence Security Audit |
| | | tx.origin Authentication Security Audit |
| 8 | Denial of Service Audit | - |
| 9 | Gas Optimization Audit | - |
| 10 | Design Logic Audit | - |
| 11 | Variable Coverage Vulnerability Audit | - |
| 12 | "False Top-up" Vulnerability Audit | - |
| 13 | Scoping and Declarations Audit | - |
| 14 | Malicious Event Log Audit | - |
| 15 | Arithmetic Accuracy Deviation Audit | - |
| 16 | Uninitialized Storage Pointer Audit | - |

# 3 Project Overview

## 3.1 Project Introduction

This is the token distribution contract for the edgeX protocol. It enables the project team to facilitate multiple distribution rounds by configuring distinct Merkle Roots for each. Users can subsequently claim their allocated tokens by presenting valid Merkle Proofs corresponding to the respective rounds.

## 3.2 Vulnerability Information

The following is the status of the vulnerabilities found in this audit:

| NO | Title | Category | Level | Status |
|---|---|---|---|---|
| N1 | Missing Zero-Address Validation for Token in Initialization Function | Design Logic Audit | Suggestion | Acknowledged |
| N2 | Missing Merkle Root Validation During Round Activation | Design Logic Audit | Suggestion | Acknowledged |
| N3 | Missing Cross-Chain Replay Protection in Leaf Nodes | Replay Vulnerability | Suggestion | Acknowledged |
| N4 | Missing Zero-Value Validation for Token Claim Amount | Design Logic Audit | Suggestion | Acknowledged |
| N5 | Missing Deduplication and Validity Validation in Off-Chain Address Parsing | Others | Low | Acknowledged |
| N6 | Missing uint256 Boundary Validation for Off-Chain Amount Parsing | Others | Suggestion | Acknowledged |

# 4 Code Overview

## 4.1 Contracts Description

**Audit Version:**

edgex-contracts.zip: 051f3ea7789af698745fead88bdacfe58aad407ca4a94e695cc160951af9a8b2

**Audit Scope:**

- merkle-distributor/src/buildTree.js
- src/distributor/EdgeDistributor.sol

The main network address of the contract is as follows:

**The code was not deployed to the mainnet.**

## 4.2 Visibility Description

The SlowMist Security team analyzed the visibility of major contracts during the audit, the result as follows:

| EdgeDistributor | | | |
|---|---|---|---|
| Function Name | Visibility | Mutability | Modifiers |
| <Constructor> | Public | Can Modify State | - |
| initialize | External | Can Modify State | initializer |
| setRound | External | Can Modify State | onlyOwner |
| setRoundActive | External | Can Modify State | onlyOwner |
| setBlacklist | External | Can Modify State | onlyOwner |
| batchClaimFor | External | Can Modify State | - |
| claimFor | External | Can Modify State | - |
| verifyClaim | External | - | - |

| EdgeDistributor | | | |
|---|---|---|---|
| withdrawUnclaimed | External | Can Modify State | onlyOwner |
| _canClaim | Internal | - | - |
| _authorizeUpgrade | Internal | Can Modify State | onlyOwner |

## 4.3 Vulnerability Summary

### [N1] [Suggestion] Missing Zero-Address Validation for Token in Initialization Function

**Category: Design Logic Audit**

**Content**

In the initialize function of the EdgeDistributor contract—which serves as the initialization entry point for the proxy—the primary responsibility is to establish the contract owner and bind the token address designated for distribution. The function fails to perform a zero-address (address(0)) validation when assigning the input _token parameter to the state variable token. Since the contract logic provides no subsequent mechanism to modify the token address, an erroneous assignment of a zero address during the deployment or initialization phase would render all core token transfer and distribution functionalities permanently inoperable. Such a configuration failure cannot be rectified within the current contract instance, necessitating a complete redeployment of the protocol.

Code location: src/distributor/EdgeDistributor.sol#L42-L45

```
function initialize(address _token, address _owner) external initializer {
    __Ownable_init(_owner);
    token = IERC20(_token);
}
```

**Solution**

It is recommended to incorporate a rigorous non-zero address check for the _token parameter prior to its assignment to the state variable.

**Status**

Acknowledged

### [N2] [Suggestion] Missing Merkle Root Validation During Round Activation

**Category: Design Logic Audit**

**Content**

In the setRoundActive function of the EdgeDistributor contract, which is primarily responsible for toggling the activation status of specific distribution rounds, the implementation fails to verify whether a valid merkleRoot has been pre-configured for the round being activated. If an administrator prematurely activates a round that has not yet had its data initialized via setRound—due to procedural oversight or operational error—the contract will enter an inconsistent state. This inadvertently exposes an invalid "active" round to the system.

Code location: src/distributor/EdgeDistributor.sol#L53-L56

```
function setRoundActive(uint256 roundId, bool active) external onlyOwner {
    rounds[roundId].active = active;
    emit RoundStatusChanged(roundId, active);
}
```

**Solution**

Incorporate a validation check for the merkleRoot prior to updating the active status.

**Status**

Acknowledged

### [N3] [Suggestion] Missing Cross-Chain Replay Protection in Leaf Nodes

**Category: Replay Vulnerability**

**Content**

Within the EdgeDistributor contract, the logic employed to generate Merkle leaf nodes only encapsulates the user's address and the claimable amount. This hash generation process omits both the current chainID and the contract's

own address. Should the project team deploy this contract across multiple chains and utilize the same Merkle Root for token distributions, a user could leverage a valid Merkle Proof obtained on one chain to initiate a replay attack on the corresponding contract on other chains. This would allow for the illicit duplication of token reward claims.

Code location: src/distributor/EdgeDistributor.sol#L89,L122

```
    function claimFor(uint256 roundId, address account, uint256 amount, bytes32[]
calldata proof) external {
        ...
        bytes32 leaf = keccak256(bytes.concat(keccak256(abi.encode(account,
amount))));
        ...
    }

    function _canClaim(uint256 roundId, address account, uint256 amount, bytes32[]
calldata proof)
        internal
        view
        returns (bool)
    {
        ...
        bytes32 leaf = keccak256(bytes.concat(keccak256(abi.encode(account,
amount))));
        ...
    }
```

**Solution**

If the protocol is intended for multi-chain deployment, it is highly recommended to incorporate the chainID and the contract address into the hash calculation to achieve robust domain separation.

**Status**

Acknowledged

### [N4] [Suggestion] Missing Zero-Value Validation for Token Claim Amount

**Category: Design Logic Audit**

**Content**

In the claimFor function of the EdgeDistributor contract—which is responsible for verifying Merkle Proofs and distributing tokens for the corresponding rounds—the implementation fails to validate whether the provided amount parameter is greater than zero prior to executing transfers and state updates. Should a proof with an amount of zero be presented, the contract will proceed to execute a zero-value token transfer and trigger a redundant Claimed event, leading to unnecessary gas consumption and cluttered event logs.

Code location: src/distributor/EdgeDistributor.sol#L83,L113

```
    function claimFor(uint256 roundId, address account, uint256 amount, bytes32[]
calldata proof) external {
        ...
    }

    function _canClaim(uint256 roundId, address account, uint256 amount, bytes32[]
calldata proof)
        internal
        view
        returns (bool)
    {
        ...
    }
```

**Solution**

Incorporate a rigorous non-zero validation for the claim amount at the function's entry point.

**Status**

Acknowledged

### [N5] [Low] Missing Deduplication and Validity Validation in Off-Chain Address Parsing

**Category: Others**

**Content**

In the standardLeafHash function within buildTree.js—a script utilized for off-chain Merkle Tree construction—the logic for extracting addresses for hashing is limited to lowercase conversion and the removal of the "0x" prefix.

Crucially, the implementation lacks both deduplication and formal format validation for the input address list. Given that the EdgeDistributor smart contract restricts each address to a single successful claim per round, if the off-chain data source contains multiple entries for the same address, only the first claim will be processed on-chain, rendering subsequent proofs for that address invalid. Furthermore, the inclusion of malformed address strings (incorrect length or invalid characters) will culminate in the generation of a dysfunctional Merkle Root.

Code location: merkle-distributor/src/buildTree.js#L28

```
function standardLeafHash(address, amount) {
  ...
  const addrHex = address.toLowerCase().replace("0x", "");
  for (let i = 0; i < 20; i++) {
    buf[12 + i] = parseInt(addrHex.slice(i * 2, i * 2 + 2), 16);
  }
  ...
}
```

**Solution**

It is recommended to implement a robust data sanitization layer during the external parsing phase, prior to invoking standardLeafHash or constructing the tree. This should include verifying that addresses conform to a 40-character hexadecimal regex (preferably with checksum validation) and performing an aggregation operation—such as summing the amount for each unique address—to consolidate duplicate entries.

**Status**

Acknowledged

### [N6] [Suggestion] Missing uint256 Boundary Validation for Off-Chain Amount Parsing

**Category: Others**

**Content**

In the standardLeafHash function of buildTree.js, which is responsible for converting allocated amounts into byte sequences for hashing, the code utilizes BigInt(amount) to parse the value and stores it in a 32-byte big-endian

buffer. However, the implementation fails to verify whether the amount overflows the maximum range of a Solidity uint256 or whether the amount is a valid non-negative integer. While the probability of encountering values exceeding 256 bits in actual distribution data is minimal, any data tampering or unexpected character parsing could result in the generation of leaf nodes that deviate from the intended total reward distribution.

Code location: merkle-distributor/src/buildTree.js#L32-L36

```
function standardLeafHash(address, amount) {
  ...
  let amt = BigInt(amount);
  for (let i = 0; i < 32; i++) {
    buf[63 - i] = Number(amt & 0xffn);
    amt >>= 8n;
  }
  ...
}
```

**Solution**

It is recommended to implement rigorous numerical assertions and range checks both before and after the BigInt conversion to confirm that the amount is a safe, non-negative integer within the permissible bounds.

**Status**

Acknowledged

# 5 Audit Result

| Audit Number | Audit Team | Audit Date | Audit Result |
|---|---|---|---|
| 0X002603300001 | SlowMist Security Team | 2026.03.30 - 2026.03.30 | Passed |

Summary conclusion: The SlowMist security team uses a manual and SlowMist team's analysis tool to audit the project, during the audit work we found 1 low risk, and 5 suggestions. All the findings were acknowledged. The code

was not deployed to the mainnet.

# 6 Statement

SlowMist issues this report with reference to the facts that have occurred or existed before the issuance of this report, and only assumes corresponding responsibility based on these.

For the facts that occurred or existed after the issuance, SlowMist is not able to judge the security status of this project, and is not responsible for them. The security audit analysis and other contents of this report are based on the documents and materials provided to SlowMist by the information provider till the date of the insurance report (referred to as "provided information"). SlowMist assumes: The information provided is not missing, tampered with, deleted or concealed. If the information provided is missing, tampered with, deleted, concealed, or inconsistent with the actual situation, the SlowMist shall not be liable for any loss or adverse effect resulting therefrom. SlowMist only conducts the agreed security audit on the security situation of the project and issues this report. SlowMist is not responsible for the background and other conditions of the project.

**Official Website**

www.slowmist.com

**E-mail**

team@slowmist.com

**Twitter**

@SlowMist_Team

**Github**

https://github.com/slowmist